

wrf_gpu: An Open-Source JAX-Native WRF v4 Port with Whole-State GPU Residency

Preprint categories: physics.ao-ph primary; cs.LG secondary; cs.SE tertiary.

Authors: Claude Opus 4.7 (AI system); GPT-5.5 Codex / OpenAI (AI system); Enric R.G. (human senior corresponding author).

Abstract

We present `wrf_gpu` v0.0.1, a source-open Python/JAX/XLA reimplementation of a WRF-compatible regional forecast path that keeps the high-frequency forecast state resident on a single consumer-grade GPU. On an NVIDIA RTX 5090 workstation the dycore achieves per-step bitwise parity with unmodified WRF v4 to 100 coupled steps; the operational forecast loop performs zero host-device transfers; the 1 hour Canary d02 pipeline is bitwise reproducible across independent runs at the same commit. Earlier WRF GPU work, including Michalakes 2008-class CUDA dynamics kernels, the Mielikainen physics-kernel series, OpenACC and OpenMP offload studies, the restricted-source WRFg line, and the proprietary AceCAST product, informs and bounds the contribution; we therefore do not claim the first GPU-enabled WRF. Preliminary skill comparison against in-situ AEMET station observations across three complete Canary d02 days shows the v0.0.1 GPU forecast is currently materially less skilful than CPU WRF (T2 +161 % to +378 % relative RMSE; U10 +214 % to +370 %; V10 +177 % to +353 %); the remaining defects are localised to surface-flux coupling and theta-guard saturation. The implementation was engineered by a frontrunner-critic-feedback multi-agent process with proof-object discipline. Code, data manifests, proof objects, and the methodology log are released openly.

1. Introduction

The Weather Research and Forecasting model is one of the standard workhorses of regional numerical weather prediction. The ARW core gives operational centers, universities, and private forecasting groups a mature nonhydrostatic model with a large ecosystem of physics options, nesting support, diagnostics, and user knowledge [1, 2]. That maturity is precisely why WRF became such an attractive GPU target and also why it has resisted a clean public GPU-native port. WRF is not a compact stencil benchmark. It is a coupled regional model whose useful behavior depends on staggered-grid dynamics, split-explicit acoustic integration, terrain-following coordinates, lateral boundary relaxation, cloud and boundary-layer physics, land state, radiation cadence, I/O, and restart semantics.

GPU attempts go back almost two decades. Michalakes and Vachharajani showed that important WRF kernels could run nearly an order of magnitude faster on GPUs, while the whole-model effect was much smaller [3]. The SSEC/NOAA family of CUDA papers accelerated WSM5, Stony Brook microphysics, Goddard shortwave radiation, Kessler, WDM6, WSM6, thermal diffusion, and YSU PBL kernels, often with very large per-scheme numbers against serial CPU baselines [4, 5, 6, 7, 8, 9, 10, 11]. Other work attacked scalar advection, horizontal diffusion, and hybrid CUDA/OpenMP routes [12, 13, 14, 15]. NVIDIA/NCAR OpenACC work, WRFg, and Summit-era OpenACC studies demonstrated that fuller

model paths could be moved toward GPUs, but they also exposed the data-transfer and restricted-source limits of that route [16, 17, 18]. More recent OpenMP offload and independent OpenACC patch repositories show that WRF GPU modernization remained active and hard into 2024-2026 [19, 20, 21].

The history is not a story of prior groups missing an obvious trick. The mathematical and software constraints are real. ARW uses an Arakawa C grid, dry hydrostatic pressure mass coordinate, RK3 outer steps, smaller acoustic substeps, and vertically coupled pressure/geopotential work that cannot be reduced to one uniform tensor update. Physics schemes such as Thompson microphysics, MYNN boundary-layer closure, RRTMG radiation, and Noah/Noah-MP land state carry branch-heavy column logic and empirical limiters [22, 23, 24, 25]. A partial GPU port can be fast in the accelerated scheme and still slow or wrong as a forecast if each coupling point copies fields back to the host, skips a physics tendency, changes a guard, or loses boundary-state timing.

This paper reports a different artifact: `wrf_gpu`, a source-open Python/JAX/XLA implementation of a WRF-compatible regional forecast path with whole-state device residency in the high-frequency loop [26, 27]. The strict novelty bound used here is: "first full open-source JAX/Python WRF v4 port with whole-state device residency on a consumer-grade workstation GPU." The word "first" is bounded by that exact definition. Prior WRF GPU work includes high-speed CUDA physics kernels, OpenCL/OpenACC and OpenMP offload studies, the restricted-source WRFg line, and the proprietary AceCAST product. We therefore do not claim the first GPU-enabled WRF. Our contribution is a source-open, WRF-compatible Python/JAX/XLA regional replay prototype that keeps the high-frequency forecast state resident on one workstation GPU and ties every performance claim to validation proof objects.

The second contribution is methodological. Earlier single-model attempts in this project, using GPT 5.4 alone and Opus 4.6 alone in separate runs, did not get close to a trustworthy port (Enric R.G., personal communication, 2026). The successful attempt used a governed frontrunner-critic-feedback workflow: a manager wrote contracts and preserved long-horizon state; worker agents implemented scoped changes; tester and reviewer agents reran commands, challenged numerical and performance claims, and blocked completion when proof objects were missing. The strongest evidence for the method is not that the agents produced a fast number. It is that the same workflow caught the original 156.82x speedup overclaim, corrected the apples-to-apples denominator, carried later skill evidence into the manuscript, and settled on the slower but defensible 22.26x current result.

What this work does not claim. v0.0.1 does not claim to be the first GPU-enabled WRF: prior CUDA physics-kernel work (Michalakes & Vachharajani 2008; Mielikainen 2012-2015), OpenACC/OpenMP offload studies, the restricted-source WRFg line, and the proprietary AceCAST product all precede it. v0.0.1 does not claim skill equivalence with CPU WRF v4: the prototype is currently materially less skilful than CPU WRF on the small Canary validation corpus reported here, and we say so transparently in Results. v0.0.1 does not claim formally conservative total energy, fully validated community-benchmark idealized-case fidelity, or deep savepoint parity beyond 100 coupled steps; those evidence categories are work for v0.1. What v0.0.1 does claim is the existence and functioning, on a single consumer-grade GPU workstation, of a source-open Python/JAX/XLA reimplementa-
tion of a WRF-compatible regional forecast path with the high-frequency forecast

state resident on the GPU, validated by per-step bitwise savepoint parity against unmodified WRF v4 at the 100-step depth.

2. Background and Related Work

WRF's ARW dynamical core solves compressible nonhydrostatic equations in flux form on a terrain-following dry hydrostatic pressure coordinate ^[1]. Its appeal is practical: a large user community, flexible physics menus, nested regional domains, and decades of operational practice. Its porting difficulty is also practical. Registry-generated state, Fortran modules, MPI decomposition, OpenMP tiling, host-side scheme control, and different array conventions create many ways for a GPU implementation to appear complete while still returning to the CPU at a coupling boundary.

The WRF-specific GPU record can be grouped into five lines. First, early CUDA studies proved kernel-level opportunity but not a complete open model ^[3, 4, 5, 6, 7, 8, 9, 10, 11]. Second, OpenCL, CUDA, and hybrid integration studies showed that whole-model acceleration drops sharply when transfers and unported work remain ^[12, 13, 14, 15]. Third, directive and WRFg work moved broader dynamics and physics paths onto accelerators but did not produce a fully source-open modern WRF v4 artifact with open validation evidence ^[16, 17, 18]. Fourth, AceCAST is a serious proprietary commercial WRF acceleration product and prevents any claim that commercial WRF GPU acceleration does not exist ^[28, 29]. Fifth, current OpenMP offload and OpenACC patch projects remain partial or scheme-focused ^[19, 20, 21].

Non-WRF GPU weather systems provide important context but not direct prior art for this claim. Pace/FV3 uses Python with GT4Py and DaCe for a different dynamical core ^[30, 31, 32, 33]. ICON-exclaim and operational ICON GPU work show what national-weather-center GPU migration can look like ^[34, 35]. SCREAM demonstrates a C++/Kokkos exascale climate-atmosphere route ^[36]. NIM is an earlier native-GPU atmospheric model ^[37]. These systems are valuable comparators because they show that production GPU NWP is possible. They do not answer whether WRF's ARW and operational regional workflow can be represented as an open JAX/XLA, workstation-scale, proof-object-validated implementation.

Machine-learning weather models are another comparator class. GraphCast, Pangu-Weather, FourCastNet, GenCast, Aurora, NeuralGCM, Stormer, and AIFS show that learned global forecast operators can be very fast and increasingly skillful ^[38, 39, 40, 41, 42, 43, 44, 45, 46]. `wrf_gpu` is not an ML emulator. It is a numerical regional forecast path that may later support differentiable parameterization work, ML boundary forcing, or hybrid learning. The distinction matters because an emulator can bypass the WRF porting problem. This paper is about that porting problem.

The AI-methodology literature is relevant because the implementation was not written by a conventional single human developer team. Repository-level coding agents are now evaluated on real software tasks by SWE-bench and SWE-agent ^[47, 48]. Multi-agent conversation systems, self-refinement, and Reflexion-style feedback show that generation followed by explicit critique can outperform one-shot generation in some settings ^[49, 50, 51]. Actor-critic ideas supply a useful analogy for separating proposal and evaluation, although this project does not claim a formal reinforcement-learning system ^[52]. The

project-specific claim is narrower: proof-object-driven AI orchestration can help build and falsify a nontrivial scientific-software artifact when the repository itself records the contracts, evidence, and rejections.

3. The Code: Architecture

The central architectural choice is whole-state device residency. In the operational loop, the high-frequency forecast state is represented as JAX arrays and remains on the GPU after initialization. Dry-air mass, perturbation pressure, geopotential, staggered winds, moisture species, selected physics tendencies, lateral side-history buffers, and surface-coupling fields are all part of the device-resident state. Output, restart writing, and validation savepoints are allowed at named boundaries. Host-device transfer inside the compiled timestep loop is not allowed. The publication audit reports zero inter-kernel D2H bytes inside the forecast loop, and the paper treats that as an architectural invariant, not a tuning detail.

The state container is deliberately plain. It is not a Fortran registry clone and it is not a black-box neural tensor bundle. Each field has a named role and a known staggering convention, and the code distinguishes prognostic state, tendencies, boundary side histories, and validation-only scratch. This is important for review: a future worker can ask why a field lives in operational carry and trace that answer to a proof object or a physics interface. It is also important for memory behavior. A single dictionary of anonymous arrays would be easy to pass through JAX, but it would make it too easy to smuggle validation-only intermediates into production or to lose track of which arrays must survive restart.

The grid follows WRF's useful physical interfaces rather than WRF's source layout. Prognostic mass-point variables and staggered velocity components are represented with explicit shape conventions. The measured 3 km d02 case uses mass shape `(44, 66, 159)` and WRF staggered extent `(45, 67, 160)`. The forecast step is 10 s, giving 360 RK steps per forecast hour. The operational timestep is expressed with `jax.lax.scan`, so the repeated forecast loop becomes an XLA program rather than a Python loop around small kernels. This matters for both speed and correctness. A Python loop would make it too easy for diagnostics, conversions, or host callbacks to enter the hot path. A scan exposes the repeated recurrence to the compiler and makes transfer auditing tractable.

The scan body is designed around timestep-scale fusion, not around maximally fusing every mathematical expression into one unreadable function. WRF's algorithm has natural recurrence boundaries: RK stages, acoustic substeps, boundary updates, physics tendencies, and output boundaries. The JAX implementation keeps those boundaries visible in source where they help validation, but expresses the repeated loops as staged computations so XLA can lower them without Python dispatch. Debug builds can expose intermediate fields for comparison; operational builds avoid diagnostic host callbacks inside the loop. That split is one of the main reasons the same codebase can support both WRF savepoint parity and zero-transfer operational timing.

The implementation preserves ARW structure at the operator level. It does not attempt a line-by-line Fortran translation. The slow meteorological step and the faster acoustic substeps are represented in Python/JAX, with validation wrappers around selected boundaries. Staggering and map-factor handling are explicit, so mass, `u`, `v`, and `w` are not treated as interchangeable dense tensors. The single-GPU v0.0.1 code carries a halo placeholder

and boundary pack/unpack structure but makes no multi-GPU scaling claim. Future MPI rank-per-GPU exchange can build on that interface; the present paper only claims the workstation single-GPU result.

Vertical coupling is the hardest part of the dycore to explain because it is not a simple horizontal stencil. Horizontally, many tendencies can be viewed as finite-volume or finite-difference updates over neighboring cells. Vertically, the pressure and acoustic recurrences involve column dependencies, scratch fields, and coefficient relationships inherited from WRF's small-step structure. The port therefore treats vertical-solve behavior as validation-sensitive. The v0.0.1 code does not present a new numerical scheme; it presents a JAX expression of the WRF-compatible recurrence with savepoint evidence at the reported depth. Faster vertical solvers, such as parallel cyclic reduction or more aggressive batched variants, are future optimization choices and would need their own Tier-1 and Tier-4 evidence.

Boundary forcing is deliberately described as a replay path. The measured d02 pipeline starts from retained Gen2 WRF products and side histories rather than from a live raw-AIFS ingest. AIFS remains the planned source for future IC/BC production work [45, 46], but v0.0.1 isolates the port by replaying a known regional case. This is why the paper calls the artifact a WRF-compatible regional replay prototype rather than a complete replacement for an operational WRF nesting system. The replay design is a constraint, not a weakness to hide: it lets the port exercise the dycore, boundary state, physics path, NetCDF writing, restart, and station verification on real Canary domains without simultaneously solving every upstream ingest problem.

The boundary representation also illustrates the difference between a tensor demo and a WRF-compatible path. Regional WRF does not run in a periodic box. The specified and relaxation zones carry external information into the domain at defined times and with defined spatial width. The iteration-2 path moved from an earlier width-1 simplification to a WRF-ordered five-row strip. That change did not close station skill, but it removed a known architectural shortcut. The paper keeps that negative outcome visible because it is useful evidence: making the boundary representation more faithful exposed, rather than solved, the remaining low-level surface/near-surface coupling problem.

Validation mode and operational mode are intentionally separated. Validation mode can emit savepoints, use stricter precision, and carry WRF scratch fields needed to compare against unmodified Fortran. Operational mode drops validation-only emission, keeps the hot state resident, and must preserve zero in-loop transfer. This separation follows from the project rule that a savepoint harness is a validation tool, not the production memory layout. A sprint that adds a field to the operational carry needs evidence that the field is required for the forecast envelope or for a named physics interface. A field needed only for comparison belongs in validation mode.

That split keeps the code from confusing instrumentation with architecture. Many scientific ports accumulate diagnostic branches that are harmless during a one-off test but fatal to a production performance claim. In `wrf_gpu`, savepoint emission, HDF5 or NetCDF comparison work, and host-side diagnostic summaries are validation-boundary operations. They are not allowed to run in the compiled hot loop used for timing. Conversely, operational mode is not allowed to remove a numerically load-bearing guard merely because validation mode can explain it. The theta and microphysics guards are ex-

amples: they are not elegant, but current evidence says they are part of the safe operational path until a source-backed replacement is validated.

The savepoint ladder is the reason the port-first claim is credible. Earlier in the project, hour-scale RMSE comparisons were not enough to localize dycore failures. The project shifted to WRF small-step instrumentation and built parity from coefficients, tridiagonal solve behavior, scratch state, acoustic recurrence, full dycore step, and coupled step. In v0.0.1, per-step bitwise parity is demonstrated to 100 coupled steps on the column savepoint tier. That does not prove 24 h forecast skill. It proves that the lower-level WRF-oriented comparison harness is real and that the dycore claim is not based solely on station scores.

The savepoint design also constrains prose claims. A statement such as "the dycore matches WRF" is too broad unless it names a depth, field set, mode, and comparator. The paper therefore says 100 coupled steps on the column savepoint tier, not unlimited equivalence. This is intentionally conservative. It gives reviewers a concrete target to reproduce or challenge, and it prevents the lower-level success from being misused as a substitute for the failed station-skill gate.

Precision is fail-closed. Hydrostatic mass, pressure-gradient-sensitive paths, and parity-sensitive operations use FP64 where the validation history requires it. FP32 or other downcasts are allowed only where policy and evidence permit them. The current implementation does not claim an aggressively mixed-precision production scheme. JAX/XLA makes it easy to change dtype or fuse operators; the project rules make those choices conditional on savepoint, invariant, or Tier-4 evidence. This is especially important for a weather model because many failures are finite and plausible for several hours before becoming meteorologically wrong.

This policy creates a useful tension with GPU performance. The fastest code would downcast broadly, fuse aggressively, and remove guards. The most conservative validation code would carry every WRF scratch field in FP64 and emit frequent savepoints. The v0.0.1 architecture keeps those as two modes with a shared core rather than pretending there is no tradeoff. The performance result is therefore not a maximum possible number. It is a measured number for the current evidence-respecting path.

Restart and I/O are also part of the architecture. The code writes hourly wrfout-style NetCDF products for the release subset and can checkpoint and restart the JAX State. The restart proof object reports max delta 0.0 for the compared state fields. This is not just an operations feature. It constrains the state representation: fields that are needed for deterministic continuation must be serializable, reloadable, and numerically stable across the restart boundary. A forecast code that cannot restart cleanly is not a serious WRF-compatible operational path.

The NetCDF writer is intentionally scoped. It provides the release subset needed for the measured pipeline and station verification, not a complete promise of every WRF diagnostic variable. That choice keeps the release honest. A future compatibility matrix can expand field coverage, metadata conventions, and restart products. For v0.0.1, the important point is that the forecast path can produce readable hourly outputs and that those outputs are connected to the same validation and skill-scoring harness used for CPU WRF comparison.

4. The Code: Physics

The v0.0.1 physics path implements a minimum operational subset aligned with common WRF choices: Thompson-style microphysics, MYNN-style boundary-layer mixing, RRTMG radiation cadence, and Noah/Noah-MP-like surface state behavior [22, 23, 24, 25]. The phrase "minimum operational subset" is deliberate. The code does not claim to reproduce every WRF physics option, every namelist branch, or every empirical limiter in the full Fortran model. It claims that selected dynamics and physics components are present in a GPU-resident regional forecast path and that their current evidence and gaps are documented.

The physics suite is currently the main source of the skill gap. Earlier defects in the pipeline were concrete. A guard path reset `theta`, `mu`, `mu_total`, and `mu_perturbation` to pre-step values, discarding prognostic RK updates. Surface fluxes were computed but not passed into MYNN bottom-boundary arrays. Radiation cadence was effectively disabled by an overly large default, so RRTMG did not run in the 24 h path. Those defects were fixed in later sprints: `theta` and `mu` now flow through RK3 with bounded guards, surface fluxes are wired into the PBL interface, and radiation cadence is 180 steps, meaning 48 RRTMG calls in a 24 h integration.

Iteration 2 removed two additional blockers from the earlier paper draft. The lateral boundary pack was widened to WRF's `spec_bdy_width=5` structure, and land-state fields such as `t_skin`, `SST`, `SMOIS`, `SH20`, and `TSLB` are refreshed hourly from retained Gen2 wrfouts at output boundaries. These changes make the pipeline more faithful to the retained WRF side-history path, but they did not close skill. The three-day AEMET comparison shows that the GPU forecast remains materially less skilful than CPU WRF for T2, U10, and V10. The likely remaining defects are a surface-flux magnitude or sign-coupling issue and `theta`-guard saturation behavior rather than a missing radiation source or a discarded RK advance.

This distinction matters for release interpretation. The dycore is validated by savepoint parity at the reported depth; the operational forecast still fails the station-skill comparison. Those statements are compatible. A regional forecast is the coupled result of dynamics, physics, boundary state, surface exchange, and output interpretation. The code architecture can be the main artifact while the physics skill gap remains the headline v0.1 objective.

5. Methodology: Multi-Agent Engineering

The implementation was produced through a governed multi-agent workflow rather than a single assistant session. The manager agent wrote sprint contracts, preserved cross-sprint memory, summarized proof objects, and decided when a result was ready for independent review. Worker agents implemented scoped changes and were limited by file ownership. Tester agents reran commands, checked artifacts, and compared claims to evidence. Reviewer and critic agents looked for missing proof, wrong denominators, unsupported physics claims, and architectural drift. The human principal set the operational target, supplied the Gen2 WRF context, made scope decisions, and remains accountable for final scientific and publication acceptance.

The contract pattern was the practical unit of work. A sprint contract named the objective, non-goals, allowed files, forbidden files, validation commands, proof objects, report token, and branch. A worker could not declare completion by saying that a command appeared to work. It had to leave a report and evidence on disk. A representative contract excerpt had this form:

```
objective:
  - isolate CPU d02-only timing from existing Gen2 records
  - compare GPU and CPU wrfout outputs against AEMET stations
acceptance:
  - emit honest_speedup_table.json
  - emit gpu_vs_cpu_skill_diff.json
hard_rules:
  - no fresh CPU WRF runs
  - taskset -c 0-3
  - be ruthless with the speedup denominator
failure_gate:
  - amend M7 closeout if speedup < 4x or GPU skill is materially worse
```

This kind of contract is intentionally narrow. It gives the worker a target small enough to complete, while giving the tester and reviewer a checklist that can fail. It also protects the repository from cross-agent collisions. Governance files, rules, memory, and goal files are not ordinary implementation files. They can only change through the patch protocol. That rule is tedious, but it prevents an agent from rewriting the acceptance criteria after failing them.

Proof objects are ordinary repository files: JSON timing tables, profiler summaries, comparison outputs, Markdown verdicts, command logs, and closeout documents. They are cited in this paper because they are the real audit surface. For example, `d2h_audit_v2.json` is the proof object for zero inter-kernel D2H; `determinism_repeat.json` is the proof object for the three-run bitwise-repeatability result; `canary_multiday_skill.json` is the proof object for the three-day skill table. Chat summaries are not proof objects. A claim that cannot be traced to a file is either removed or marked as a limitation.

The project also treats proof objects as typed evidence. A timing JSON can support a wall-clock claim, but not a physics claim. A restart comparator can support restart continuity, but not station skill. A savepoint result can support a local WRF-parity statement, but not a 24 h operational replacement claim. Several project errors came from crossing those wires. The current paper is organized to keep the evidence type next to the claim it supports.

The methodology's most important success was self-correction. The original M7 closeout reported 156.82x speedup. A later sprint found that the timing denominator double-counted the wrong CPU timing records and that GPU-only station scores had been overread as operational skill. The paper then moved through 50.20x for the pre-fix diagnostic path and finally to 22.26x for the current iteration-2 path after additional physics, boundary, and land-state changes. The correction did not make the result less valuable. It made it publishable: the number now matches the claim and the skill regression is visible in the abstract, results, limitations, and discussion.

Cross-model review mattered because the disagreements were concrete. One model could argue that a performance claim was plausible; another could demand the CPU de-

nominator path. One model could explain a likely physics coupling bug; another could run a bisection and return a JSON table. The manager did not accept disagreement as a philosophical tie. It converted disagreement into the next sprint's file ownership and pass/fail gate. That is the practical version of the frontrunner-critic-feedback loop.

The first failed attempt also shaped the workflow. In project history, GPT 5.4 alone and Opus 4.6 alone each failed to get close to a trustworthy WRF GPU port (Enric R.G., personal communication, 2026). The failure mode was not simply lack of code generation ability. A single model tended to over-commit to one explanation, lose track of validation obligations, or accept a plausible numerical story without adversarial pressure. The later frontrunner-critic-feedback loop made disagreement routine. One role could build, another could attack, and a manager could turn the failed hypothesis into the next contract.

The limits of the method are equally important. The manager was an AI system and made a publication-facing error before the later audit caught it. The tester and reviewer roles reduced risk but did not replace an independent human numerical-methods review. The workflow also depends on strong contracts: if the contract asks for finite station scores rather than side-by-side GPU-vs-CPU skill, a worker can complete the wrong proxy faithfully. This paper treats that as a method result. The process improved because the failed proxy was preserved and then corrected, not because the process was perfect from the beginning.

This process does not make the agents authorities. It creates friction. It forces every performance claim to name a denominator, every physics claim to cite a comparison or limitation, and every milestone closeout to survive review. It also leaves a trail of wrong turns, which is essential in numerical software. The paper therefore presents the multi-agent workflow as a reproducible engineering method under human responsibility, not as autonomous scientific certification.

6. Validation Strategy

The project uses a four-tier validation strategy. Tier 1 is local parity: analytic fixtures and WRF savepoints with explicit shapes, units, staggering, and tolerances. Tier 2 is physical invariant checking: finite values, basic bounds, dry-mass behavior, tracer positivity, and water-budget diagnostics where physics is active. Tier 3 is short-run trajectory behavior: does the coupled system remain controlled over early steps and expected perturbations? Tier 4 is operational statistical evidence: grid and station comparisons, ensemble consistency, or verification metrics on real cases. Tier 1 can prove that an operator reproduces a WRF comparison point. Tier 4 can reject an operational claim. Neither replaces the other.

The v0.0.1 evidence is intentionally mixed. Savepoint parity is strong at the reported depth. D2H transfer evidence is strong inside the forecast-loop window. Restart and one-hour determinism are bitwise. Conservation and stability evidence are operational surrogates rather than formal community-benchmark gates. Forecast skill is negative against CPU WRF on the small three-day station corpus. A paper that hides the negative Tier-4 result would be less rigorous than a paper that reports it and limits the claim.

The validation stack also separates correctness from speed. A GPU run can be fast, finite, and repeatable while still failing skill. The pre-fix path demonstrated exactly that. Its wall time looked impressive, but later evidence showed missing or miswired physics behavior.

The present paper therefore uses performance evidence only after systems invariants and limitations are named. The result is a port-first paper, not an operational-skill paper.

The planned v0.1 validation extensions are clear. Community benchmark idealized cases, formal closed-domain conservation, deeper savepoint parity, a larger Canary side-by-side sample, precipitation verification, and surface-flux repair are all listed as future work. None of those items is treated as done in v0.0.1.

7. Results

7.1 Architecture and Execution Evidence

The publication-test harness was re-run end-to-end on a healthy NVIDIA RTX 5090 (32 607 MiB total, about 26 200 MiB used at sprint time) ^[53], consuming 1.226 GPU-hours across the HIGH-priority set (`aggregate_report.json`). Four Canary d02 forecasts were executed (20260428 partial-history 2 h, 20260509 24 h, 20260521 24 h, 20260525 24 h) with forecast wall-clock between 572 s and 713 s for the complete-day cases.

The operational forecast loop preserves the residency invariant. The D2H audit reports 0 copies and 0 bytes of inter-kernel device-to-host transfer inside the compiled forecast loop. Warm one-hour d02 timing remains about 5.71 s in the measured profile window, and the one-step 1 km feasibility probe reports 7278 MiB used out of 32607 MiB on the RTX 5090. The memory number is not a full 1 km forecast validation or peak allocator trace; it is a feasibility result for state shape and one warmed step.

The current 24 h performance result is the iteration-2 path. It is slower than the pre-fix diagnostic path and slightly slower than iteration 1, but it is the correct publication number because it includes the currently accepted physics, boundary, and land-refresh changes.

System state	Metric	Value	Proof object
Iteration-2 current path	24 h d02 pipeline wall time	732.63 s	.agent/sprints/2026-05-27-m7-skill-fix- iter2/pipeline_run_20260521.json
Iteration-2 current path	24 h forecast- only wall time	687.90 s	.agent/sprints/2026-05-27-m7-skill-fix- iter2/pipeline_run_20260521.json
Iteration-2 current path	Apples-to-apples d02-only speed- up	22.26x	.agent/sprints/2026-05-27-m7-skill-fix- iter2/post_iter2_speedup.json
Iteration-1 predecessor	24 h d02 pipeline wall time	708.32 s	.agent/sprints/2026-05-27-m7-skill-fix-al- gorithmic/pipeline_run_20260521.json
Iteration-1 predecessor	Apples-to-apples d02-only speed- up	23.02x	.agent/sprints/2026-05-27-m7-skill-fix-al- gorithmic/post_fix_speedup.json
Pre-fix dia- gnostic path	24 h d02 pipeline wall time	324.78 s	.agent/sprints/2026-05-27-m7-daily-pipeline- integration/pipeline_run_20260521.json
Pre-fix dia- gnostic path	Apples-to-apples d02-only speed- up	50.20x	.agent/sprints/2026-05-27-m7-honest-speedup- skill-diff/honest_speedup_table.json
Rejected closeout claim	Original head- line speedup	156.82x rejected	.agent/decisions/MILESTONE-M7-CLOSEOUT- AMENDMENT.md

7.2 Determinism, Savepoint Parity, Conservation, and Stability

Under identical inputs, commit, and environment, three independent 1-hour Canary d02 pipeline runs produce bitwise-identical wrfout files across all 41 archived fields (max absolute delta = 0 for every field; total recorded GPU runtime 17.6 s for the three runs). The proof object is `determinism_repeat.json` at `.agent/sprints/2026-05-27-testing-plan-execution-redo/`.

Per-step bitwise parity against unmodified WRF v4 is demonstrated to 100 coupled steps on the column savepoint tier of the Canary d02 case (`savepoint_deep_column100.json`, outcome SEVENTH-COUPLED-STEP-PARITY-ACHIEVED extended to step 100). The revised-plan stretch depths of 1000 and 10000 coupled steps are deferred to v0.1.

Operational dry-mass behaviour on the 24 h Canary d02 forecast for 2026-05-21 shows maximum relative drift of 4.81e-6 (uncorrected for boundary flux), below the revised-plan threshold of 1e-5 for the boundary-flux-corrected residual; a theta-and-geopotential proxy energy diagnostic over the same forecast bounds total relative drift at 3.09 % over 24 h (`conservation_mass_24h.json`, `conservation_energy_24h.json`).

Operational stability is supported by Canary d02 1-hour surrogates at dt in {0.5x, 1.0x, 1.25x nominal} and acoustic-substep counts in {4, 6, 8}: all six runs produce finite output, with pairwise surface nRMSE (T2, U10, V10) bounded by 4.16e-3 across acoustic-substep settings (`stability_cfl_sweep.json`, `stability_acoustic_substep.json`).

7.3 Forecast Skill

The forecast-skill result is negative. Against in-situ AEMET station observations ^[54], on the same valid times in the Canary d02 domain, none of T2, U10, V10 is within +/-20 % of the CPU WRF RMSE on any of the three complete days. The partial-history 20260428 case is excluded because it has zero valid joined station pairs.

Day	Variable	CPU RMSE (m/s or K)	GPU RMSE	Relative delta
2026-05-09	T2 (K)	2.51	11.97	+378 %
2026-05-09	U10 (m/s)	2.12	7.21	+240 %
2026-05-09	V10 (m/s)	2.21	6.51	+195 %
2026-05-21	T2 (K)	2.15	10.80	+303 %
2026-05-21	U10 (m/s)	2.31	7.24	+214 %
2026-05-21	V10 (m/s)	2.75	7.62	+177 %
2026-05-25	T2 (K)	2.95	7.71	+161 %
2026-05-25	U10 (m/s)	2.11	9.92	+370 %
2026-05-25	V10 (m/s)	2.24	10.16	+353 %

The skill gap is reported as part of the result, not hidden in limitations. The most likely current explanation is a surface-flux magnitude or sign-coupling issue interacting with theta-guard saturation. The dycore savepoint result narrows the likely location of the remaining defect, but it does not excuse the operational skill failure.

8. Canary Case Study

The Canary Islands d02 case is a representative development workload, not the paper's identity. It was chosen because it is small enough for a single RTX 5090 but meteorologically demanding: steep volcanic terrain, trade-wind flow, island wakes, marine boundary-layer structure, sharp surface contrasts, and strong sensitivity to low-level heat and momentum exchange. A simple periodic box would not have stressed lateral boundaries, surface coupling, restart, wrfout writing, and station verification in the same way.

The current release uses four Canary dates in the publication-test corpus: 20260428 as a 2 h partial-history case and 20260509, 20260521, and 20260525 as complete 24 h forecasts. The three complete days are sufficient to show that the pipeline runs across multiple retained cases and that the skill gap is not a single failed file. They are not sufficient to make a climatological or seasonal statement. The planned v0.1 work is to extend the side-by-side corpus after Gen2 history backfill and then rerun the station and grid comparisons.

The case study also explains why the paper separates artifact success from forecast skill. The same system can demonstrate whole-state residency, restart continuity, 100-step savepoint parity, and fast 24 h execution while still over-warming the low-level atmosphere. For an operational meteorologist, the current answer is simple: this is not yet a WRF skill replacement. For a scientific-computing reader, the current answer is also useful: a source-open JAX/XLA WRF-compatible port now exists with enough validation machinery to localize and repair the remaining defects.

9. Discussion

The port-first result changes what is available to the community. A WRF-compatible regional forecast path can now be inspected in Python/JAX, compiled by XLA, run on a consumer GPU, and audited against WRF-oriented proof objects. That opens lines of work that are awkward in legacy WRF: differentiable sensitivity studies, rapid parameterization experiments, JAX-native ML coupling, workstation-scale ensemble economics, and direct inspection of compiled forecast graphs. These possibilities are not operational claims. They are the reason a source-open port matters even before v0.1 closes the skill gap.

Whole-state residency is the architectural lesson. Many prior WRF GPU efforts found that per-kernel acceleration did not survive whole-model integration because host/device movement, unported physics, or host-side control remained in the loop. `wrf_gpu` takes the opposite route: define the high-frequency state as device resident and make any transfer exception explicit. The zero-D2H loop audit is therefore more important than any one timing number. It says the code has crossed from "accelerated component" toward "GPU-native forecast loop."

The 22.26x current speedup is still meaningful after the correction from 156.82x. It is measured against the d02-only 28-rank CPU WRF denominator on the same workstation, and it includes the current iteration-2 boundary and land-refresh path. The pre-fix 50.20x result remains useful as a warning. It proves that a faster path can be less valid. A paper about numerical software should preserve that warning instead of optimizing the story around the biggest number.

The multi-agent process is not a guarantee of correctness, but it improved the repository's ability to reject bad claims. The manager itself made the original closeout error. The value of the process was that later contracts forced the error into the open: first by checking the denominator, then by comparing GPU and CPU skill side by side, then by carrying the negative result into the manuscript. That is a stronger methodology claim than saying that agents "built code." The method built code and then changed the published claim when the evidence required it.

The main scientific risk is now narrower than it was before the fix sprints. The problem is not that no GPU WRF path exists, or that radiation is absent, or that the RK update is discarded. The known current problem is that the coupled surface/near-surface physics path produces materially worse station RMSE than CPU WRF. Closing that gap is the v0.1 headline. The likely fix path is to audit surface-flux sign and magnitude, theta guard saturation, land-state prognostic behavior, and MYNN bottom-boundary coupling under the same proof-object rules.

10. Open Source Release Plan

The public release target is `github.com/wrf-gpu/wrf_gpu` under AGPL-3.0. The release package should include source code, `INSTALL.md`, a runnable example, citation guidance, proof-object manifests, and contribution instructions. Binary fixture data remains outside git under the documented external-data policy; tracked manifests and small sample slices identify what was used. The manager repository remains the integration and governance home during sprint work, and the public repository receives staged release artifacts after review.

Versioning follows the claim boundary. The v0.0.x line is the honest prototype release: source-open, JAX/XLA, GPU-resident, fast, reproducible at the reported depths, but not skill-equivalent to CPU WRF. The v0.1.0 line is reserved for the arXiv companion and the first post-gap release after the known validation gaps are addressed or explicitly re-baselined. The 1.0.0 version is reserved for an operational claim, which requires skill closure, larger verification, restart and I/O hardening, and independent review.

Contributions should follow the same proof-object discipline as the internal work. A pull request that changes dynamics, physics, precision, transfer behavior, or performance claims should include the relevant fixture comparison, invariant check, transfer audit, profiler artifact, or forecast-skill comparison. The release should make that expectation explicit so external users do not treat the repository as a black-box demo.

11. Limitations

L6. The v0.0.1 GPU forecast is currently materially less skilful than CPU WRF on station-observation comparison. Across the three complete Canary days reported here, GPU T2 RMSE is +161 % to +378 % versus CPU WRF, GPU U10 RMSE +214 % to +370 %, and GPU V10 RMSE +177 % to +353 %. The defects appear localised to surface-flux coupling and theta-guard saturation behaviour rather than the dycore: the dycore passes per-step bitwise WRF parity to 100 coupled steps and the pipeline is bitwise reproducible end-to-end. Closing this skill gap is the headline v0.1 objective.

L1. Community-benchmark idealized cases deferred to v0.1. The Bryan & Fritsch (2002) warm-bubble, Straka et al. (1993) density-current, and Schaer et al. (2002) sinusoidal-terrain mountain-wave idealized cases are not validated against published references in v0.0.1: the analytic initial-condition builders are present and finite-checked, but reviewed GPU integrators for the three cases are deferred. Dycore correctness in v0.0.1 is carried instead by step-by-step bitwise savepoint parity against unmodified WRF v4 (M6b6 seven-step result and the 100-step column-tier extension in this release).

L2. Conservation evidence is operational, not formal. Closed-domain warm-bubble dry-mass drift at $\leq 1e-10$ and Tier-4 total-energy envelope against a CPU WRF reference are deferred to v0.1; v0.0.1 reports operational evidence only (uncorrected Canary 24 h dry-mass drift $4.81e-6$, well below the $1e-5$ corrected-residual threshold; proxy total-energy drift bounded at 3.09 % over 24 h).

L3. CFL and acoustic-substep evidence uses an operational surrogate. The revised-plan warm-bubble CFL sweep and density-current acoustic-substep sweep are not run in v0.0.1; Canary d02 1 h surrogates at Δt in $\{0.5x, 1.0x, 1.25x\}$ and acoustic-substep in $\{4, 6, 8\}$ are reported instead, all finite, with pairwise surface nRMSE $\leq 4.16e-3$ across acoustic settings.

L4. Savepoint parity demonstrated to 100 coupled steps in v0.0.1. Bitwise WRF-parity is reported at 7 coupled steps (M6b6 baseline) and 100 coupled steps (this release, column tier). The 1000- and 10000-step depth gates are stretch targets from the revised plan; they are not in v0.0.1 scope and are deferred to v0.1.

L5. Canary side-by-side covers 4 days, not the originally planned 14. v0.0.1 reports Canary d02 comparison on 4 days (20260428, 20260509, 20260521, 20260525), of which

3 are complete 24 h forecasts and one is a 2-hour partial-history case. The original-plan ≥ 14 -day window is deferred to v0.1 once Gen2 history backfill completes.

L7. Determinism is demonstrated on a 1-hour Canary d02 segment. Three independent 1-hour pipeline runs on identical inputs and commit produce bitwise-identical wrfout files across all 41 archived fields. Full 24-hour pipeline determinism is expected to hold under the same deterministic XLA kernels but is not separately demonstrated in v0.0.1.

Additional limitations follow from the release boundary. The code is single-GPU only; the halo path is an interface placeholder, not a scaling result. The 1 km memory result is a one-step feasibility probe, not a full 1 km forecast validation. The current workflow is re-play-driven from retained Gen2 WRF products rather than live AIFS ingestion. Finally, the project has not yet had an independent human numerical-methods audit of the full code and manuscript. Human review remains necessary before any operational adoption.

12. Reproducibility

The current reproducibility package is file-oriented. The paper's claims are tied to proof objects in `.agent/decisions/`, `.agent/sprints/`, and `publication/draft/honesty_audit.md`. The lightweight audit command is:

```
taskset -c 0-3 bash scripts/m7_publication_audit.sh
```

The audit checks manuscript word count, ASCII text, BibTeX parseability, cited-key integrity, required proof-object existence, and AgentOS validation. It does not rerun GPU forecasts or CPU WRF. Heavy reruns depend on the RTX 5090, retained Gen2 data, and external data paths.

Environment field	Value
Python	3.13.11
JAX	0.10.0
jaxlib	0.10.0
CUDA toolkit	13.1.115
NVIDIA driver	595.71.05
GPU	NVIDIA GeForce RTX 5090, 32607 MiB
OS	Linux 6.17.0-29-generic x86_64
CPU pinning for publication checks	<code>taskset -c 0-3</code>

Key proof objects include `.agent/sprints/2026-05-27-testing-plan-execution-redo/determinism_repeat.json`, `.agent/sprints/2026-05-27-testing-plan-execution-redo/savepoint_deep_column100.json`, `.agent/sprints/2026-05-27-testing-plan-execution-redo/canary_multiday_skill.json`, `.agent/sprints/2026-05-27-testing-plan-execution-redo/conservation_mass_24h.json`, `.agent/sprints/2026-05-27-testing-plan-execution-redo/conservation_energy_24h.json`, `.agent/sprints/2026-05-27-testing-plan-execution-redo/stability_cfl_sweep.json`, `.agent/sprints/2026-05-27-testing-plan-execution-redo/stabil-`

ity_acoustic_substep.json, .agent/sprints/2026-05-27-m7-skill-fix-iter2/post_iter2_speedup.json, and .agent/decisions/MILESTONE-M7-CLOSEOUT-AMENDMENT.md.

13. Author Contributions and AI Use Disclosure

Claude Opus 4.7 is identified as an AI system contributor. It managed sprint contracts, preserved long-horizon repository context, synthesized proof objects, drafted and revised manuscript text, and made the original closeout overclaim that later sprints corrected. GPT-5.5 Codex / OpenAI is identified as an AI system contributor. It implemented scoped repository changes, performed debugging and critical-review work, generated proof objects under sprint contracts, and contributed empirical evidence used in the paper.

Enric R.G. defined the Canary operational target, supplied and maintained the Gen2 CPU WRF baseline context, set validation and performance expectations, supervised the AI-agent process, and retains senior corresponding-author responsibility for final scientific acceptance and submission. All external publication responsibility rests with the human author. The AI systems cannot approve the final manuscript, hold legal accountability, or satisfy human-only authorship criteria.

This authorship framing is policy-sensitive. The briefed arXiv discussion emphasizes author responsibility for unchecked AI-generated content ^[55, 56], while publisher policies such as Nature's do not treat AI tools as authors ^[57]. For an arXiv preprint, the current byline is a transparent AI-contribution disclosure. If a target venue requires only human authors, the byline should be changed to Enric R.G. alone, with Claude Opus 4.7 and GPT-5.5 Codex / OpenAI moved to acknowledgements plus this AI-use disclosure.

14. Acknowledgements

The project depends on the WRF and NCAR modeling community, ECMWF AIFS context, AEMET station observations, NVIDIA GPU tooling, and Enric R.G.'s prior Gen2 operational Canary forecasting system. The authors also acknowledge the repository's internal tester, reviewer, critic, and manager roles, which forced the correction from the original performance celebration to the current proof-object-backed reporting.

References

1. William C. Skamarock, Joseph B. Klemp, Jimy Dudhia, David O. Gill, Zhiqian Liu, Judith Berner, Wei Wang, Jordan G. Powers, Michael G. Duda, Dale M. Barker, Xiang-Yu Huang. (2019) "A Description of the Advanced Research WRF Model Version 4." National Center for Atmospheric Research https://www2.mmm.ucar.edu/wrf/users/docs/technote/v4_tech-note.pdf
2. Jordan G. Powers, Joseph B. Klemp, William C. Skamarock, Christopher A. Davis, Jimy Dudhia, David O. Gill, Michael G. Duda, Xiang-Yu Huang, Wei Wang, John B. Klemp. (2017) "The Weather Research and Forecasting Model: Overview, System Efforts, and Future Directions." *Bulletin of the American Meteorological Society* 98(8) pp. 1717--1737 doi: [10.1175/BAMS-D-15-00308.1](https://doi.org/10.1175/BAMS-D-15-00308.1)
3. John Michalak, Manish Vachharajani. (2008) "GPU Acceleration of Numerical Weather Prediction." *Parallel Processing Letters* 18(4) pp. 531--548 doi: [10.1142/S0129626408003557](https://doi.org/10.1142/S0129626408003557)
4. Jarno Mielikainen, Bormin Huang, Hung-Lung Allen Huang, Mitchell D. Goldberg. (2012) "Improved GPU/CUDA Based Parallel Weather and Research Forecast (WRF) Single Moment 5-Class (WSM5) Cloud Microphysics." *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 5(4) pp. 1256--1265 doi: [10.1109/JSTARS.2012.2188780](https://doi.org/10.1109/JSTARS.2012.2188780)
5. Jarno Mielikainen, Bormin Huang, Hung-Lung Allen Huang, Mitchell D. Goldberg. (2012) "GPU Implementation of Stony Brook University 5-Class Cloud Microphysics Scheme in the WRF." *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 5(2) pp. 625--633 doi: [10.1109/JSTARS.2011.2175707](https://doi.org/10.1109/JSTARS.2011.2175707)
6. Jarno Mielikainen, Bormin Huang, Hung-Lung Allen Huang, Mitchell D. Goldberg. (2012) "GPU Acceleration of the Updated Goddard Shortwave Radiation Scheme in the Weather Research and Forecasting (WRF) Model." *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 5(2) pp. 555--562 doi: [10.1109/JSTARS.2012.2186119](https://doi.org/10.1109/JSTARS.2012.2186119)
7. Jun Wang, Bormin Huang, Antonio J. Plaza, Hung-Lung Allen Huang, Mitchell D. Goldberg. (2013) "Compute Unified Device Architecture (CUDA)-Based Parallelization of WRF Kessler Cloud Microphysics Scheme." *Computers & Geosciences* 52 pp. 292--299 doi: [10.1016/j.cageo.2012.10.006](https://doi.org/10.1016/j.cageo.2012.10.006)
8. Jarno Mielikainen, Bormin Huang, Hung-Lung Allen Huang, Mitchell D. Goldberg, Ajay Mehta. (2013) "Speeding Up the Computation of WRF Double-Moment 6-Class Microphysics Scheme with GPU." *Journal of Atmospheric and Oceanic Technology* 30(12) pp. 2896--2906 doi: [10.1175/JTECH-D-12-00218.1](https://doi.org/10.1175/JTECH-D-12-00218.1)
9. Melin Huang, Bormin Huang, Lingjia Gu, Hung-Lung Allen Huang, Mitchell D. Goldberg. (2015) "Parallel GPU Architecture Framework for the WRF Single Moment 6-Class Microphysics Scheme." *Computers & Geosciences* 83 pp. 17--26 doi: [10.1016/j.cageo.2015.06.014](https://doi.org/10.1016/j.cageo.2015.06.014)
10. Melin Huang, Bormin Huang, Yang-Lang Chang, Jarno Mielikainen, Hung-Lung Allen Huang, Mitchell D. Goldberg. (2015) "Efficient Parallel GPU Design on WRF Five-Layer Thermal Diffusion Scheme." *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 8 pp. 2249--2259 doi: [10.1109/JSTARS.2015.2422268](https://doi.org/10.1109/JSTARS.2015.2422268)
11. Melin Huang, Jarno Mielikainen, Bormin Huang, H. Chen, Hung-Lung Allen Huang, Mitchell D. Goldberg. (2015) "Development of Efficient GPU Parallelization of WRF Yonsei University Planetary Boundary Layer Scheme." *Geoscientific Model Development* 8 pp. 2977--2990 doi: [10.5194/gmd-8-2977-2015](https://doi.org/10.5194/gmd-8-2977-2015)

12. Wim Vanderbauwhede, Tetsuya Takemi. (2016) "An Analysis of the Feasibility and Benefits of GPU/Multicore Acceleration of the Weather Research and Forecasting Model." *Concurrency and Computation: Practice and Experience* 28(7) pp. 2052--2072 doi: [10.1002/cpe.3522](https://doi.org/10.1002/cpe.3522)
13. Ronald M. Gualan-Saavedra, Lizandro D. Solano-Quinde, Brett M. Bode. (2015) "GPU Acceleration of the Horizontal Diffusion Method in the Weather Research and Forecasting (WRF) Model." In *Proceedings of the 2015 Asia-Pacific Conference on Computer-Aided System Engineering* pp. 284--289 doi: [10.1109/APCASE.2015.57](https://doi.org/10.1109/APCASE.2015.57)
14. Juan Pablo Silva, Jose Hagopian, Marcel Burdiat, Ernesto Dufrechou, Martin Pedemonte, Alejandro Gutierrez, Gabriel Cazes, Pablo Ezzatti. (2014) "Another Step to the Full GPU Implementation of the Weather Research and Forecasting Model." *The Journal of Supercomputing* 70(2) pp. 746--755 doi: [10.1007/s11227-014-1193-y](https://doi.org/10.1007/s11227-014-1193-y)
15. Ridwan Ridwan, Achmad Imam Kistijantoro, Mahally Kudsy, Djoko Gunawan. (2015) "Performance Evaluation of Hybrid Parallel Computing for WRF Model with CUDA and OpenMP." In *2015 3rd International Conference on Information and Communication Technology* pp. 425--430 doi: [10.1109/ICoICT.2015.7231463](https://doi.org/10.1109/ICoICT.2015.7231463)
16. NVIDIA. (2016) "GPU Developments for the WRF Model." https://www2.mmm.ucar.edu/wrf/users/workshops/WS2016/oral_presentations/4.4.pdf
17. Jeff Adie, Carl Ponder, Alexey Romanenko, Stan Posey. (2018) "GPU Developments in Atmospheric Sciences: WRF Current GPU Status." https://www2.mmm.ucar.edu/wrf/users/workshops/WS2018/oral_presentations/5.1.pdf
18. Gokhan Sever, Jeff Adie, Rajeev Jain, Stan Posey, Charlie Catlett. (2019) "Performance Evaluation of the Weather Research and Forecasting (WRF) Model on the DOE Summit Supercomputer." https://www2.mmm.ucar.edu/wrf/users/workshops/WS2019/oral_presentations/3.3.pdf
19. Chayanon Wichitrnithed, Woo-Sun Yang, Yun He, Brad Richardson, Koichi Sakaguchi, Manuel Arenaz, William I. Gustafson Jr., Jacob Shpund, Ulises Costi Blanco, Alvaro Goldar Dieste. (2024) "Optimizing the Weather Research and Forecasting Model with OpenMP Offload and Codee." In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis* pp. 1934--1942 doi: [10.1109/SCW63240.2024.00243](https://doi.org/10.1109/SCW63240.2024.00243)
20. FahrenheitResearch. (2026) "WRF 4.7.1 GPU Port (OpenACC)." <https://github.com/FahrenheitResearch/wrf-gpu-port>
21. WRF, MPAS-A Support Forum. (2026) "WRF GPU Port (FahrenheitResearch)." <https://forum.mmm.ucar.edu/threads/wrf-gpu-port-fahrenheitresearch.26779/>
22. Gregory Thompson, Paul R. Field, Roy M. Rasmussen, William D. Hall. (2008) "Explicit Forecasts of Winter Precipitation Using an Improved Bulk Microphysics Scheme. Part II: Implementation of a New Snow Parameterization." *Monthly Weather Review* 136(12) pp. 5095--5115 doi: [10.1175/2008MWR2387.1](https://doi.org/10.1175/2008MWR2387.1)
23. Mikio Nakanishi, Hiroshi Niino. (2006) "An Improved Mellor-Yamada Level-3 Model: Its Numerical Stability and Application to a Regional Prediction of Advection Fog." *Boundary-Layer Meteorology* 119 pp. 397--407 doi: [10.1007/s10546-005-9030-8](https://doi.org/10.1007/s10546-005-9030-8)
24. Michael J. Iacono, Jennifer S. Delamere, Eli J. Mlawer, Mark W. Shephard, Shepard A. Clough, William D. Collins. (2008) "Radiative Forcing by Long-Lived Greenhouse Gases: Calculations with the AER Radiative Transfer Models." *Journal of Geophysical Research: Atmospheres* 113(D13) pp. D13103 doi: [10.1029/2008JD009944](https://doi.org/10.1029/2008JD009944)
25. Guo-Yue Niu, Zong-Liang Yang, Kenneth E. Mitchell, Fei Chen, Michael B. Ek, Michael Barlage, Anil Kumar, Kevin Manning, Dev Niyogi, Enrique Rosero, Mukul Tewari, Youlong Xia. (2011) "The Community Noah Land Surface Model with Multiparameterization Options

- (Noah-MP): 1. Model Description and Evaluation with Local-Scale Measurements." *Journal of Geophysical Research: Atmospheres* 116(D12) pp. D12109 doi: [10.1029/2010JD015139](https://doi.org/10.1029/2010JD015139)
26. James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin. (2018) "JAX: Composable Transformations of Python+NumPy Programs." <https://github.com/google/jax>
 27. Roy Frostig, Matthew James Johnson, Chris Leary. (2018) "Compiling Machine Learning Programs via High-Level Tracing." In *Proceedings of MLSys* <https://mlsys.org/Conferences/doc/2018/146.pdf>
 28. Gene Pache, Amirreza Rastegari. (2023) "Microsoft and TempoQuest Accelerate Wind Energy Forecasts with AceCAST." <https://developer.nvidia.com/blog/?p=64091>
 29. TempoQuest Inc.. (2025) "AceCAST Verification and Validation Technical Documentation." <https://acecast-docs.readthedocs.io/>
 30. Jeremy Dahm, Oliver Elbert, Rusty Benson, Lucas Harris, Linjiong Zhou, Katherine McInerney. (2023) "Pace: A Python-Based Performance-Portable Atmospheric Model." *Geoscientific Model Development* 16(9) pp. 2719--2736 doi: [10.5194/gmd-16-2719-2023](https://doi.org/10.5194/gmd-16-2719-2023)
 31. Samuel Whitaker, Carlos Osuna, Benjamin Cumming, Thomas Thaler. (2023) "GT4Py: A Python Framework for the Development of High-Performance Stencil Codes for Weather and Climate Applications." *Geoscientific Model Development* 16 pp. 1205--1221 doi: [10.5194/gmd-16-1205-2023](https://doi.org/10.5194/gmd-16-1205-2023)
 32. Tal Ben-Nun, Johannes de Fine Licht, Alexandros Nikolaos Ziogas, Timo Schneider, Torsten Hoefer. (2019) "Stateful Dataflow Multigraphs: A Data-Centric Model for Performance Portability on Heterogeneous Architectures." In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*
 33. Enrique G. Paredes, Linus Groner, Stefano Ubbiali, Hannes Vogt, Alberto Madonna, Kean Mariotti, Felipe Cruz, Lucas Benedicic, Mauro Bianco, Joost VandeVondele, Thomas C. Schulthess. (2023) "GT4Py: High Performance Stencils for Weather and Climate Applications Using Python." *SoftwareX* doi: [10.48550/arXiv.2311.08322](https://doi.org/10.48550/arXiv.2311.08322)
 34. Oliver Fuhrer, Xavier Lapillonne, Daniel Hupp, Fabian Gessler, Andre Walser, Andreas Pauling. (2026) "Operational Numerical Weather Prediction with ICON on GPUs (Version 2024.10)." *Geoscientific Model Development* 19 pp. 755--772 doi: [10.5194/gmd-19-755-2026](https://doi.org/10.5194/gmd-19-755-2026)
 35. Xavier Lapillonne, Oliver Fuhrer, Carlos Osuna. (2026) "Operational Benchmarks of the ICON Model on Heterogeneous GPU Systems." *Geoscientific Model Development* 19 pp. 773--789 doi: [10.5194/gmd-19-773-2026](https://doi.org/10.5194/gmd-19-773-2026)
 36. Luca Bertagna, Andrew M. Bradley, Peter M. Caldwell, Aaron Donahue, Oksana Guba, Noel D. Keen, Mark A. Taylor. (2024) "SCREAM: The Simple Cloud-Resolving E3SM Atmosphere Model Running on the Frontier Exascale System." In *Proceedings of the Platform for Advanced Scientific Computing Conference* pp. 1--12 doi: [10.1145/3620256](https://doi.org/10.1145/3620256)
 37. Mark Govett, Jim Rosinski, Jacques Middlecoff, Tom Henderson, Jin Lee, Alexander MacDonald. (2017) "Parallelization and Performance of the NIM Weather Model on CPU, GPU, and MIC Processors." *Bulletin of the American Meteorological Society* 98(10) pp. 2201--2213 doi: [10.1175/BAMS-D-15-00278.1](https://doi.org/10.1175/BAMS-D-15-00278.1)
 38. Remi Lam, Alvaro Sanchez-Gonzalez, Matthew Willson, Peter Wirnsberger, Meire Fortunato, Ferran Alet, Suman Ravuri, Timo Ewalds, Zach Eaton-Rosen, Weihua Hu, Alexander Merose, Stephan Hoyer, George Holland, Oriol Vinyals, Jacklynn Stott, Alexander Pritzel, Shakir Mohamed, Peter Battaglia. (2023) "Learning Skillful Medium-Range Global Weather Forecasting." *Science* 382(6677) pp. 1416--1421 doi: [10.1126/science.adi2336](https://doi.org/10.1126/science.adi2336)

39. Kaifeng Bi, Lingxi Xie, Hengheng Zhang, Xin Chen, Xiaotao Gu, Qi Tian. (2022) "Pangu-Weather: A 3D High-Resolution Model for Fast and Accurate Global Weather Forecast." *arXiv preprint arXiv:2211.02556*
40. Jaideep Pathak, Shashank Subramanian, Peter Harrington, Sanjeev Raja, Ashesh Chattopadhyay, Morteza Mardani, Thorsten Kurth, David Hall, Zongyi Li, Kamyar Azizzadenesheli, Pedram Hassanzadeh, Karthik Kashinath, Animashree Anandkumar. (2022) "FourCastNet: A Global Data-Driven High-Resolution Weather Model Using Adaptive Fourier Neural Operators." *arXiv preprint arXiv:2202.11214*
41. Ilan Price, Alvaro Sanchez-Gonzalez, Ferran Alet, Tom R. Andersson, Andrew El-Kadi, Dominic Masters, Timo Ewalds, Jacklynn Stott, Shakir Mohamed, Peter Battaglia, Remi Lam, Matthew Willson. (2023) "GenCast: Diffusion-Based Ensemble Forecasting for Medium-Range Weather." *arXiv preprint arXiv:2312.15796*
42. Cristian Bodnar, Wessel P. Bruinsma, Ana Lucic, Megan Stanley, Anna Vaughan, Johannes Brandstetter, Patrick Garvan, Maik Riechert, Jonathan A. Weyn, Haiyu Dong, Jayesh K. Gupta, Kit Thambiratnam, Alexander T. Archibald, Chun-Chieh Wu, Elizabeth Heider, Max Welling, Richard E. Turner, Paris Perdikaris. (2024) "A Foundation Model for the Earth System." *arXiv preprint arXiv:2405.13063*
43. Dmitrii Kochkov, Janni Yuval, Ian Langmore, Peter Norgaard, Jamie Smith, Griffin Mooers, Milan Klower, James Lottes, Stephan Rasp, Peter Duben, Sam Hatfield, Peter Battaglia, Alvaro Sanchez-Gonzalez, Matthew Willson, Michael P. Brenner, Stephan Hoyer. (2023) "Neural General Circulation Models for Weather and Climate." *arXiv preprint arXiv:2311.07222*
44. Tung Nguyen, Rohan Shah, Hritik Bansal, Troy Arcomano, Romit Maulik, Veerabhadra Kotamarthi, Ian Foster, Sandeep Madireddy, Aditya Grover. (2023) "Scaling Transformer Neural Networks for Skillful and Reliable Medium-Range Weather Forecasting." *arXiv preprint arXiv:2312.03876*
45. Simon Lang, Mihai Alexe, Matthew Chantry, Jesper Dramsch, Florian Pinault, Baudouin Raoult, Mariana C. A. Clare, Christian Lessig, Michael Maier-Gerber, Linus Magnusson, Zied Ben Bouallegue, Ana Prieto Nemesio, Peter D. Dueben, Andrew Brown, Florian Pappenberger, Florence Rabier. (2024) "AIFS: ECMWF's Data-Driven Forecasting System." *arXiv preprint arXiv:2406.01465*
46. Simon Lang, Massimo Bonavita, Zied Ben Bouallegue. (2025) "An Update to ECMWF's Machine-Learned Weather Forecasting Model AIFS." *arXiv preprint arXiv:2509.18994*
47. Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, Karthik Narasimhan. (2024) "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?." In *The Twelfth International Conference on Learning Representations* <https://openreview.net/forum?id=VTF8yNQM66>
48. John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, Ofir Press. (2024) "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering." *arXiv preprint arXiv:2405.15793* <https://arxiv.org/abs/2405.15793>
49. Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, Chi Wang. (2023) "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation." *arXiv preprint arXiv:2308.08155*
50. Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, Peter Clark. (2023) "Self-Refine: Iterative Refinement with Self-Feedback." *arXiv preprint arXiv:2303.17651*

51. Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, Shunyu Yao. (2023) "Reflexion: Language Agents with Verbal Reinforcement Learning." *arXiv preprint arXiv:2303.11366*
52. Vijay R. Konda, John N. Tsitsiklis. (2000) "Actor-Critic Algorithms." In *Advances in Neural Information Processing Systems*
53. PNY Technologies. (2025) "GeForce RTX 5090 Models: Product Specifications." <https://www.pny.com/geforce-rtx-5090-models>
54. Agencia Estatal de Meteorologia. (2026) "Climatological Summaries and Operational Meteorological Observations over Canary Islands." <https://www.aemet.es/en/portada>
55. Thomas G. Dietterich. (2026) "Public Statement on arXiv Author Responsibility for Generative AI Output." <https://techcrunch.com/2026/05/16/research-repository-arxiv-will-ban-authors-for-a-year-if-they-let-ai-do-all-the-work/>
56. PCMag. (2026) "Major Scientific Repository arXiv Cracks Down on AI-Generated Papers." <https://www.pcmag.com/news/major-scientific-repository-arxiv-cracks-down-on-ai-generated-papers>
57. Nature Portfolio. (2026) "Artificial Intelligence (AI): Scientific Data Policy." <https://www.nature.com/sdata/policies/ai>